



ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

HIAN MATHEUS DA SILVA

ANÁLISE DE DESEMPENHO DE ESTRUTURAS DE DADOS EM JAVA

MURIAÉ

2025

HIAN MATHEUS DA SILVA

ANÁLISE DE DESEMPENHO DE ESTRUTURAS DE DADOS EM JAVA

Trabalho apresentado a Faminas como requisito para avaliação da Matéria de Estrutura de Dados do curso Análise e Desenvolvimento de Sistemas.

Professor: Flávio Andrade Amaral Motta

MURIAÉ

2025

Resumo

Este trabalho apresenta uma análise experimental detalhada do desempenho das estruturas Vetor, Árvore Binária de Busca (ABB) e Árvore AVL implementadas em Java. Os testes foram conduzidos com três tamanhos de entrada (100, 1000 e 10.000 elementos) em três padrões distintos: ordenado, inverso e aleatório. Os resultados demonstram que o Vetor apresenta tempos de inserção extremamente baixos e comportamento muito estável, independentemente da ordem de entrada, destacando-se em todos os cenários. A ABB, por outro lado, apresentou forte degradação em dados ordenados e inversos, chegando a tempos de inserção na ordem de centenas de milhões de nanossegundos para 10.000 elementos, comportamento esperado devido ao crescimento desbalanceado da árvore. A AVL, embora apresente custos de inserção mais elevados devido às rotações de balanceamento, manteve tempos de busca mais estáveis que a ABB, especialmente em grandes volumes de dados. Os resultados reforçam a relação direta entre complexidade teórica e desempenho prático, evidenciando como o padrão de entrada influencia diretamente o custo computacional das estruturas analisadas.

1.Introdução

O estudo de estruturas de dados é fundamental para compreender como diferentes organizações de informação influenciam o tempo de execução de operações básicas como inserção e busca. Em aplicações reais, o desempenho dessas operações pode variar drasticamente dependendo tanto da estrutura utilizada quanto do padrão dos dados de entrada. Assim, testar e comparar o comportamento dessas estruturas em cenários controlados é essencial para fundamentar escolhas adequadas em desenvolvimento de software e análise algorítmica.

Neste trabalho, analisamos o desempenho experimental de três estruturas clássicas: Vetor, Árvore Binária de Busca (ABB) e Árvore AVL. Para cada uma delas, foram avaliados os tempos de inserção e busca utilizando três tamanhos de entrada (100, 1000 e 10.000 elementos) e três padrões de organização dos dados: ordenado, inverso e aleatório. As medições foram feitas utilizando `System.nanoTime()`, com repetição de operações para reduzir ruídos estatísticos.

Os resultados obtidos revelam comportamentos distintos e alinhados às complexidades teóricas. O Vetor apresentou excelente desempenho em todas as situações, mantendo tempos reduzidos tanto para inserção quanto para busca. A ABB demonstrou forte sensibilidade ao padrão dos dados, apresentando degradação significativa nos casos ordenado e inverso, nos quais a árvore tende a se tornar uma lista encadeada. Já a AVL, graças ao balanceamento automático, exibiu maior consistência nos tempos de busca, embora com inserções mais custosas devido ao processo de rotação.

Dessa forma, o presente estudo busca relacionar o comportamento prático observado com as previsões teóricas, oferecendo uma visão clara sobre os impactos do volume e da natureza dos dados no desempenho das estruturas analisadas.

2. Metodologia

A metodologia adotada para a análise de desempenho das estruturas de dados consistiu em quatro etapas principais: geração dos conjuntos de dados, execução dos testes, medição dos tempos e organização dos resultados. Cada etapa foi planejada de forma a garantir consistência, repetibilidade e precisão nas medições, possibilitando uma comparação justa entre Vetor, Árvore Binária de Busca (ABB) e Árvore AVL.

2.1 Geração dos Dados de Entrada

Para avaliar o impacto do padrão dos dados no desempenho das estruturas, foram utilizados três tipos de entrada:

- **Ordenada:** sequência crescente de valores inteiros.
- **Inversa:** sequência decrescente de valores.
- **Aleatória:** valores gerados com a função `Math.random()` ou equivalente, sem repetição.

Esses padrões foram escolhidos pois representam cenários clássicos que influenciam o comportamento das estruturas, especialmente das árvores, que podem se desbalancear dependendo da ordem dos elementos.

Os testes foram realizados com três tamanhos diferentes de entrada:

- **100 elementos**
- **1.000 elementos**
- **10.000 elementos**

Esses tamanhos permitem observar como o comportamento prático se aproxima (ou se afasta) da complexidade teórica conforme o volume de dados cresce.

2.2 Execução dos Testes

Para cada combinação de tamanho e padrão de entrada, foram executadas as seguintes operações:

Inserção

Cada elemento do conjunto foi inserido na estrutura correspondente. No caso da AVL, o processo inclui eventuais rotações para manter o balanceamento.

Buscas

Foram analisadas diferentes estratégias de busca para melhor observar o comportamento em diferentes posições:

- **Busca do primeiro elemento**
- **Busca do último elemento**
- **Busca de um elemento intermediário**
- **Três buscas aleatórias**
- **Busca por um elemento inexistente**

Os valores das buscas foram selecionados de forma a simular diferentes cenários de acesso, incluindo caso de sucesso e de falha.

Cada operação de busca foi executada separadamente em cada estrutura para isolar o custo individual da busca.

2.3 Procedimento de Medição

As medições foram realizadas utilizando o método:

System.nanoTime()

A escolha por nanossegundos se deve à sua maior precisão, permitindo diferenciar operações muito rápidas, como inserções em vetor e buscas bem-sucedidas.

O tempo total de cada operação foi calculado da seguinte forma:

1. **Início da medição:** antes de executar a operação.
2. **Fim da medição:** imediatamente após sua finalização.
3. **Tempo final:** diferença entre as duas medições.

Para reduzir variações externas, cada operação foi repetida várias vezes (5 repetições de acordo com o que foi pedido nas regras do trabalho) e o programa registrou a **média dos tempos**, garantindo resultados mais estáveis.

2.4 Ambiente de Execução (Configuração do Hardware)

Os testes foram executados em um notebook pessoal, cuja configuração influencia diretamente os tempos de execução obtidos. As especificações do hardware e do software utilizado são as seguintes:

- **Processador:** 11th Gen Intel® Core™ i5-1135G7 × 8
- **Memória RAM:** 8 GB
- **Armazenamento:** 256 GB
- **Sistema Operacional:** Linux Mint
- **Versão do Java utilizada (JDK):** JDK 21.0.8
- **IDE utilizada:** Vs Code e IntelliJ

A definição desse ambiente é importante para contextualizar os resultados e permitir comparação futura em hardwares diferentes.

2.5 Organização e Apresentação dos Resultados

Os tempos médios coletados foram organizados em tabelas por:

- Estrutura (Vetor, ABB, AVL)
- Padrão dos dados (Ordenada, Inversa, Aleatória)
- Tamanho da entrada (100, 1000, 10.000)

Além disso, serão apresentados gráficos comparativos que auxiliam na visualização das diferenças práticas entre:

- Tempos de inserção
- Tempos de busca

Finalmente, os resultados foram analisados relacionando desempenho prático às complexidades teóricas esperadas:

- Vetor: $O(1)$ para acesso, $O(n)$ para inserção ordenada
- ABB não balanceada: pior caso $O(n)$
- AVL: inserções e buscas em $O(\log n)$

3. Apresentação dos Resultados

Esta seção apresenta os resultados obtidos nos experimentos realizados com as três estruturas de dados analisadas: **Vetor**, **Árvore Binária de Busca (ABB)** e **Árvore AVL**. Os testes contemplaram operações de **inserção**, **busca** e, no caso do Vetor, **ordenação** utilizando os algoritmos **Bubble Sort** e **Quick Sort**.

Os resultados foram organizados conforme:

- **Tamanho da entrada:** 100, 1.000 e 10.000 elementos
- **Padrão dos dados:** ordenado, inverso e aleatório
- **Estrutura avaliada**
- **Médias dos tempos** em nanossegundos (ns)

A seguir, são apresentados e analisados os principais comportamentos observados.

3.1 Resultados para 100 elementos

Os tempos mostram que, para entradas pequenas, todas as estruturas apresentam desempenho estável. O Vetor manteve tempos muito baixos de inserção e busca. A ABB apresentou tempos maiores quando os dados estavam ordenados ou invertidos, evidenciando o desbalanceamento. A AVL manteve estabilidade mesmo em padrões adversos, como esperado.

Destaques principais

- Vetor tem melhor desempenho em todos os cenários.
- ABB sofre penalização severa para dados ordenados e inversos.
- AVL apresenta inserções mais custosas que ABB, mas com buscas mais consistentes.
- Ordenação via Bubble é muito mais lenta que Quick, mesmo para 100 elementos.

3.1.1 Tabela – Entradas Ordenadas (100)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
11609,80 0 ns	444,600 ns	319,000 ns	17341,600 / 631,400 / 518,800 ns	281,400 ns	6101,600 ns	150155, 600 ns	113418, 800 nss

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
313416,20 0 ns	825,200 ns	9561,200 ns	2001,800 / 556,400 / 498,600 ns	300,400 ns	164,200 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
398594,60 0 ns	788,400 ns	596,200 ns	1755,000 / 599,600 / 487,000 ns	241,800 ns	200,200 ns

3.1.2 Tabela – Entradas Inversas (100)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
10920,20 0 ns	368,200 ns	227,800 ns	2713,600 / 692,400 / 620,400 ns	200,800 ns	7839,400 ns	259054, 600 ns	33701,8 00 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
213882,40 0 ns	7838,800 ns	409,800 ns	2863,400 / 980,200 / 1794,600 ns	326,200 ns	237,600 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
11609,800 ns	444,600 ns	319,000 ns	17341,600 / 631,400 / 518,800 ns	281,400 ns	6101,600 ns

3.1.3 Tabela - Entradas Aleatórias (100)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
12326,200 ns	429,600 ns	253,800 ns	2311,600 / 876,600 / 760,400 ns	193,000 ns	9100,200 ns	246409,200 ns	11198,400 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
22547,200 ns	747,200 ns	499,400 ns	1645,600 / 811,200 / 763,800 ns	228,400 ns	142,200 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
127706,200 ns	849,800 ns	739,800 ns	2009,000 / 1272,000 / 772,000 ns	232,600 ns	144,600 ns

3.2 Resultados para 1.000 elementos

Ao aumentar o volume de dados, os comportamentos teóricos ficam mais evidentes. A ABB apresenta crescimento explosivo nos tempos de inserção para dados ordenados, chegando a vários milhões de nanossegundos. A AVL mantém tempos maiores de inserção devido ao balanceamento, porém buscas previsivelmente rápidas.

Destaques principais

- Inserção na ABB ordenada/inversa alcança mais de **5 milhões de ns**.
- Vetor mantém buscas extremamente rápidas ($< 1 \mu s$).
- Quick Sort supera Bubble de forma crescente conforme o tamanho aumenta.
- AVL se mantém estável e previsível, validando o custo **$O(\log n)$** .

3.2.1 Tabela – Entradas Ordenadas (1000)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
88542,400 ns	1334,400 ns	516,200 ns	9114,600 / 722,400 / 2656,400 ns	338,200 ns	69255,400 ns	1921340,400 ns	575306,200 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
5162655,600 ns	3252,200 ns	72400,600 ns	5038,600 / 532,200 / 395,800 ns	1118,400 ns	317,000 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
6126383,000 ns	1584,800 ns	812,000 ns	178,000 / 262,200 / 255,800 ns	518,000 ns	193,200 ns

3.2.2 Tabela – Entradas Inversas (1000)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
78163,600 ns	707,800 ns	198,200 ns	2892,600 / 354,200 / 287,400 ns	228,400 ns	70180,200 ns	1677230,600 ns	626203,200 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
4446114,800 ns	74193,000 ns	911,000 ns	5399,000 / 545,600 / 439,200 ns	1083,400 ns	303,200 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
5285933,400 ns	1435,600 ns	639,400 ns	1591,400/210,800 / 180,600 ns	338,400 ns	88,000 ns

3.2.3 Tabela – Entradas Aleatórias (1000)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
63107,000 ns	186,200 ns	164,400 ns	3051,600 / 266,600 / 208,600 ns	141,600 ns	60707,200 ns	990595,200 ns	118791,800 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
108258,00 0 ns	651,000 ns	787,200 ns	551,600 / 246,200 / 202,600 ns	331,000 ns	107,200 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
7475779,2 00 ns	5144,800 ns	1750,000 ns	9784,000/ 339,200 / 181,800 ns	1607,200 ns	920,200 ns

3.3 Resultados para 10.000 elementos

Com 10 mil elementos, as diferenças entre as estruturas se tornam marcantes. A ABB em cenário ordenado se degrada para tempos acima de **290 milhões de nanosegundos**, indicando claramente o caso degenerado da árvore. A AVL mantém consistência nas buscas, embora o custo das inserções seja substancial devido ao balanceamento.

Destaques principais

- Vetor continua sendo a estrutura mais rápida para inserção e busca.
- Bubble Sort se torna impraticável: tempo ultrapassa **90 milhões de ns**.
- Quick Sort mantém ótimo desempenho, mesmo em 10.000 elementos.
- ABB totalmente desbalanceada causa explosão de tempo.
- **AVL comprova robustez em busca, ainda que com inserções caras.**

3.3.1 Tabela – Entradas Ordenadas (10000)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
243995,000 ns	4298,000 ns	2313,600 ns	3874,400 / 209,600 / 217,000 ns	1782,000 ns	82663,600 ns	16823601,000 ns	42707321,000 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
290933589,400 ns	9906,800 ns	341410,600 ns	3481,000 / 283,000 / 176,000 ns	2806,800 ns	920,600 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
42476778 1,600 ns	2286,800 ns	785,400 ns	1698,800 / 121,200 / 85,400 ns	598,000 ns	201,400 ns

3.3.2 Tabela – Entradas Inversas (10000)**Vetor:**

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
139539,0 00 ns	755,200 ns	290,800 ns	1016,800 / 93,000 / 86,400 ns	176,400 ns	9364,000 ns	6518942 0,800 ns	2497830 8,000 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
26168844 0,800 ns	325945,4 00 ns	444,400 ns	2102,400 / 160,200 / 136,000 ns	971,200 ns	323,400 ns

Árvore AVL:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
43516032 9,200 ns	4019,000 ns	1002,400 ns	3243,600 / 215,600 / 233,400 ns	1159,000 ns	146,200 ns

3.3.3 Tabela – Entradas Aleatórias (10000)

Vetor:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente	Bubble Sort	Quick Sort
114357,800 ns	2309,800 ns	1046,200 ns	4070,600 / 2052,600 / 74,400 ns	962,200 ns	9672,400 ns	92101741,800 ns	549200,000 ns

Árvore ABB:

Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
1070388,000 ns	1962,600 ns	1275,400 ns	879,000 / 274,000 / 73,200 ns	892,800 ns	753,800 ns

Árvore AVL:

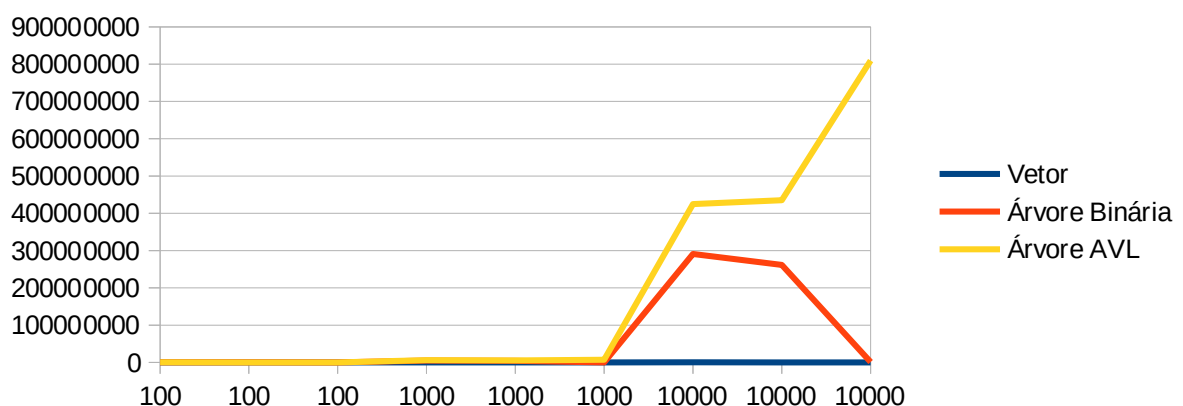
Inserção	Busca Primeiro	Busca último	Buscas aleatórias	Busca meio	Busca inexistente
809579383,800 ns	3612,800 ns	1060,800 ns	1788,600 / 1171,400 / 94,600 ns	838,000 ns	92,200 ns

4. Gráficos de Desempenho

Os gráficos abaixo ilustram o comportamento das estruturas de dados analisadas (Vetor, Árvore Binária e AVL) em termos de **tempos de inserção** e **tempos de busca**, considerando diferentes tamanhos de conjuntos (100, 1.000 e 10.000 elementos) e diferentes ordens de entrada (Ordenada, Inversa e Aleatória).

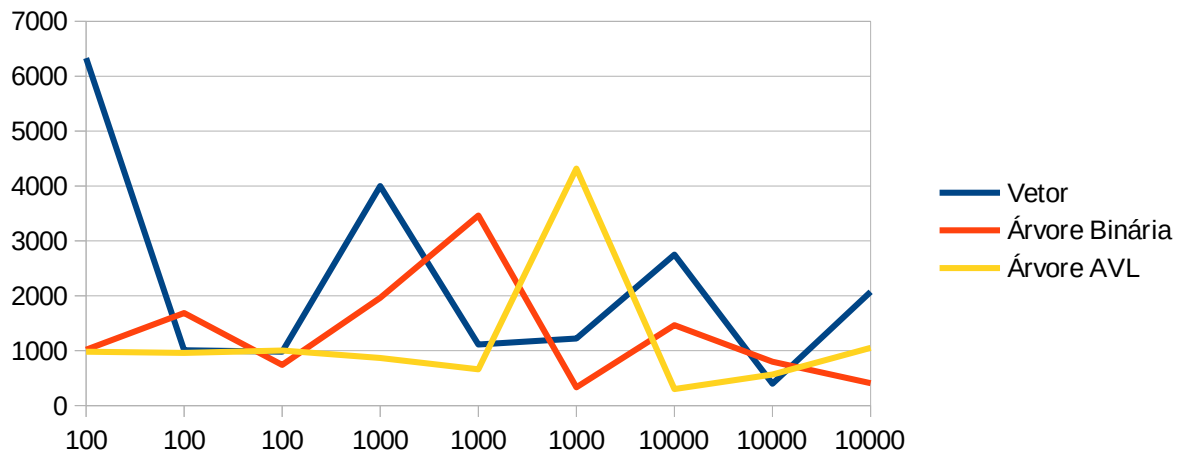
4.1 Tempos de Inserção

- Observa-se que o **Vetor** mantém tempos extremamente baixos, mesmo em conjuntos grandes.
- A **Árvore Binária (ABB)** cresce rapidamente em conjuntos grandes, especialmente para entradas ordenadas ou inversas, refletindo o caso degenerado da árvore.
- A **AVL** apresenta inserções mais custosas, mas se mantém previsível graças ao balanceamento automático.



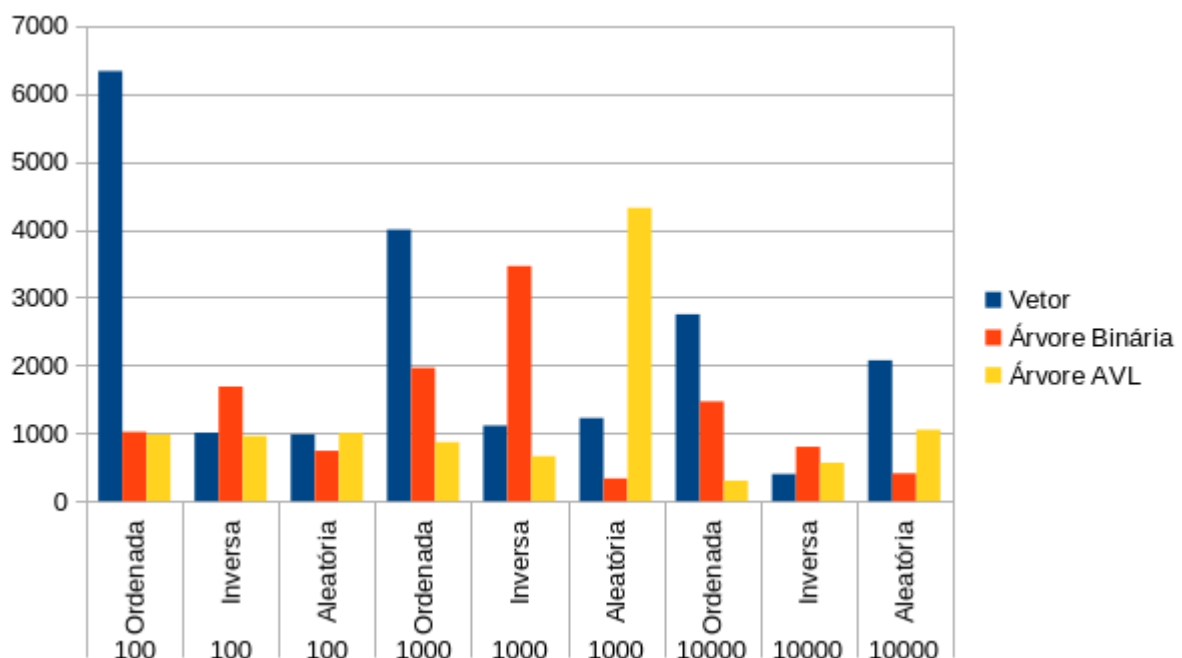
4.2 Tempos de Busca

- O **Vetor** continua apresentando buscas rápidas e consistentes
- A **ABB** tem desempenho irregular, dependendo da ordenação dos dados, com tempos elevados em conjuntos desbalanceados.
- A **AVL** mantém buscas rápidas e estáveis, mesmo em conjuntos grandes, demonstrando sua robustez.



4.3 Comparação das Estruturas – Inserção e Busca

- Observa-se que o **Vetor** mantém tempos de inserção e busca consistentemente baixos em todos os cenários, demonstrando eficiência tanto para operações de inserção direta quanto para acesso por índice.
- A **Árvore Binária de Busca (ABB)** apresenta um crescimento acentuado nos tempos de inserção para dados ordenados ou inversos, enquanto seus tempos de busca variam significativamente, refletindo a degradação da estrutura em cenários desbalanceados.
- A **Árvore AVL** mantém inserções mais custosas, porém previsíveis, e garante tempos de busca estáveis e eficientes, validando o custo do balanceamento automático para operações de consulta em grandes volumes de dados.



5. Conclusão Geral do projeto

Este trabalho apresentou uma análise comparativa do desempenho experimental de três estruturas de dados clássicas implementadas em Java: Vetor, Árvore Binária de Busca (ABB) e Árvore AVL, considerando diferentes tamanhos de entrada (100, 1.000 e 10.000 elementos) e diferentes padrões de organização (ordenado, inverso e aleatório).

Os resultados obtidos confirmam a forte relação entre a complexidade teórica e o desempenho prático das estruturas analisadas:

1. **Vetor** destacou-se como a estrutura mais eficiente tanto em inserções quanto em buscas, mantendo tempos baixos e consistentes em todos os cenários testados. Sua simplicidade e baixo custo de acesso ($O(1)$) o tornam ideal para cenários em que a ordem de inserção não é crítica e o volume de dados não exige operações de reordenação frequentes.

2. **Árvore Binária de Busca (ABB)** demonstrou forte sensibilidade ao padrão dos dados, especialmente em entradas ordenadas ou inversas, onde se degenera em uma lista encadeada, resultando em tempos de inserção extremamente elevados (centenas de milhões de nanossegundos). Esse comportamento evidencia a importância do balanceamento em aplicações práticas.

3. **Árvore AVL**, embora apresente custos de inserção mais altos devido às rotações de balanceamento, manteve tempos de busca estáveis e previsíveis, em conformidade com a complexidade $O(\log n)$. Isso comprova sua robustez e adequação para cenários em que a consistência no tempo de resposta é essencial, mesmo diante de grandes volumes de dados.

Além disso, foram observadas diferenças significativas entre os algoritmos de ordenação Bubble Sort e Quick Sort, com este último demonstrando desempenho claramente superior em todos os tamanhos testados, especialmente a partir de 1.000 elementos.

Portanto, a escolha da estrutura de dados mais adequada deve considerar não apenas a complexidade teórica, mas também o padrão e o volume dos dados de entrada. Enquanto o Vetor é uma solução eficiente e simples para conjuntos moderados, a AVL oferece garantias de desempenho em cenários dinâmicos e de grande escala, ainda que com um custo adicional de inserção. A ABB, por sua vez, só se torna viável quando há garantia de dados aleatórios ou balanceamento externo.

Este estudo reforça a importância da análise empírica como complemento à teoria, fornecendo subsídios para decisões fundamentadas no desenvolvimento de sistemas computacionais eficientes e escaláveis.